C PROGRAMMING FROM SCRATCH · EPISODE 10

Type Modifiers

Fine-Tuning the Built-In Types

KEY POINTS

- Type modifiers come in two groups: short and long (length) and signed and unsigned (sign).
- short and long change how much memory an integer may use; long also works with double.
- signed types can hold negative values; unsigned types hold only zero and positive values.
- The modifier goes in front of the type, and with int the modifier alone is enough.

01 Why modifiers?

▶ VIDEO 00:52

A cup can be described in two separate ways: how much it holds (200, 600 or 1000 ml) and what kind of cup it is (heat-resistant, food-grade). **Data types work the same way: a modifier takes a basic type and describes it more precisely.**

02 Two groups

▶ VIDEO 01:41

> NOTE **Length:** `short`, `long`
>
> How much memory a value uses. A `short int` is meant for whole numbers that fit in less memory, a `long int` for whole numbers that may need more. The exact number of bytes depends on the compiler.

> NOTE **Sign:** `signed`, `unsigned`
>
> Whether a value can be negative. A signed type holds negative numbers, zero and positive numbers; an unsigned type holds only zero and positive values.

All four words are among the 32 classic C keywords.

03 Writing modified types

▶ VIDEO 02:49

WRITE	MEANING
`short int`, `long int`, `unsigned int`, `signed char`	the modifier goes in front of the type
`unsigned long int`	both groups together: a long integer that can't be negative
`short`, `long`, `unsigned`	`short int`, `long int`, `unsigned int`
`int`	already signed: the same type as `signed int`
`long double`	`long` also works with `double`

KEY IDEA

There's no `short float`, and floating-point types can't be marked `signed` or `unsigned`.

04 In Visual Studio

▶ VIDEO 03:49

modifiers.c — C SOURCE

```c
#include <stdio.h>

int main()
{
    short int temperature = -5;
    unsigned char level = 200;
    printf("%d\n", temperature);
    printf("%d\n", level);
    return 0;
}
```

Output

```
-5
200
```

`%d` works here because smaller integer types like `short` and `unsigned char` are passed to `printf` as `int`.

WATCH OUT **short float**

Adding `short float weight = 1.5;` makes the build fail: `error C2632: 'short' followed by 'float' is illegal` (`mistakes/short-float.c`).

CODE 2 files on GitHub github.com/abitclearer/c-programming-from-scratch/tree/main/10-type-modifiers